

# Scaling Reads

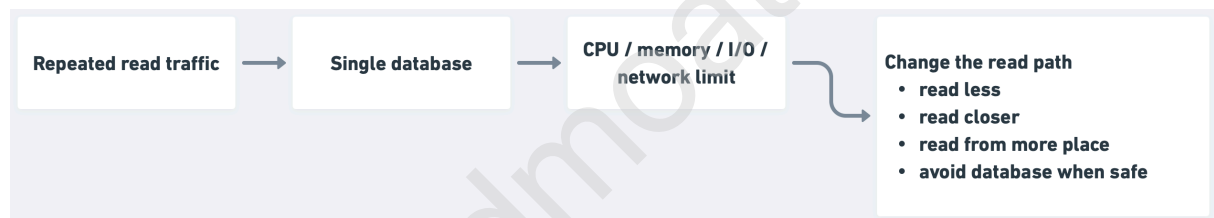
## 讀取擴展在解什麼問題？

讀取擴展（Scaling Reads）不是單純把 API 變快，而是要讓系統在大量使用者反覆讀同一批資料時，database 不會被拖垮。它處理的是「同樣的答案被問太多次」這件事。

很多課程案例都有這種不對稱。QR Code Generator 裡，一個 QR token 可能只建立一次，之後卻被掃描或 redirect 幾百萬次。Airbnb 的房源資料不會每秒改，但旺季搜尋和瀏覽會大量重複讀。YouTube 的影片 metadata、manifest file 和縮圖在上傳後相對穩定，但每次播放前都會被讀取。

讀寫比一旦拉開，系統瓶頸就會從「怎麼寫進去」變成「怎麼不要每次都回頭問同一個 database」。例如短網址 redirect 的讀寫比可能接近 100 : 1；Airbnb 旺季瀏覽可以到數十萬 QPS；影音平台的觀看請求更是遠高於影片上傳。

所以讀取擴展的核心問題是：



- 同樣的資料，被很多人反覆讀
- 同一個 endpoint，被流量大量集中
- 單一 database 的 CPU、記憶體、磁碟 I/O 或網路頻寬先到上限
- 使用者期待低延遲，但資料又不能錯太多

這不是單純「把程式寫快一點」能解的問題。當 database 每秒要掃描、排序、join 或傳輸的資料量超出硬體能力時，真正要做的是改變讀取路徑：少讀一點、近一點讀、從多個地方讀，或乾脆不要每次都讀 database。

## 先問三個診斷問題

在面試或設計系統時，不要一聽到「讀取量很高」就立刻喊 Redis。先把問題拆清楚，解法才會準。

## 讀的是什麼？

不同資料的讀取策略差很多。

- 房源詳情、公開 market data、影片 metadata：很多人會讀同一份資料，很適合 cache
- 使用者個人設定、私人訊息、帳戶資訊：幾乎只有本人會讀，cache hit rate 不一定高
- 搜尋結果、排行榜、order query view：通常是計算結果，可能更適合預先計算或 materialized view
- 庫存、餘額、交易狀態：新鮮度要求高，不能只靠長 TTL cache

讀取擴展不是把所有資料都快取，而是找出「讀很多、變很少、可共享」的資料。

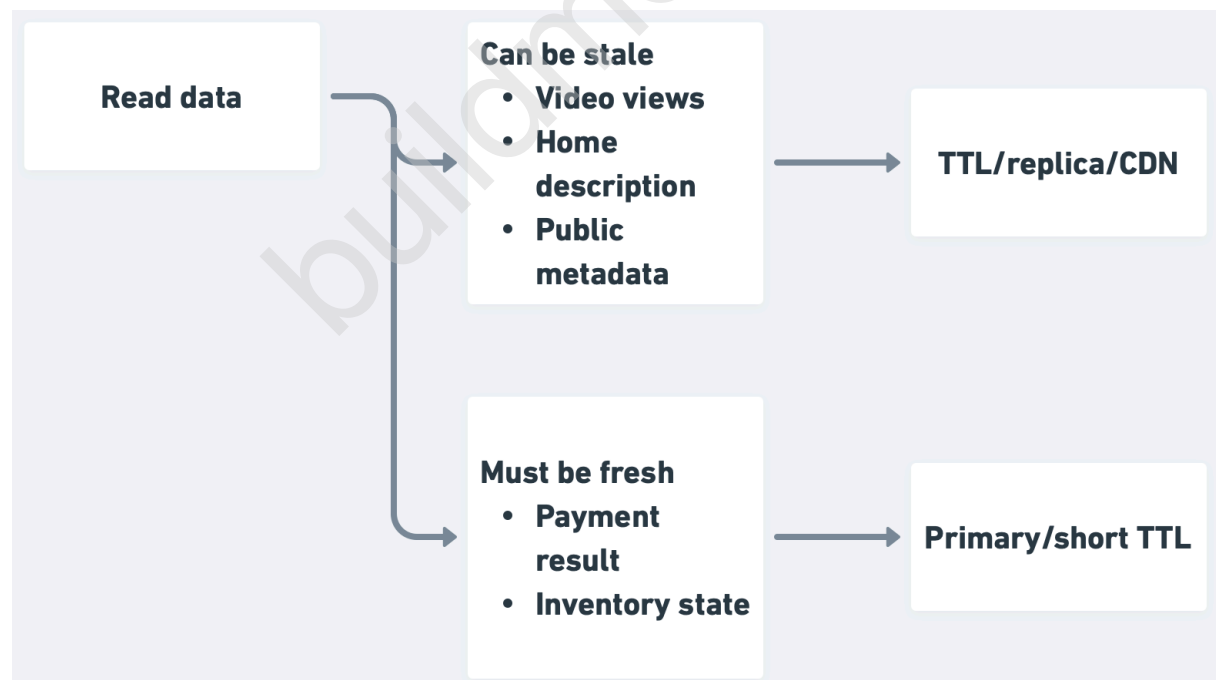
## 讀取流量怎麼分布？

流量通常不是平均分布。少數熱門 key 會吃掉大多數讀取量，這叫 access skew。

例如 QR Code Generator 裡，某個活動海報上的 QR code 可能突然成為 hot key；Airbnb 裡，熱門城市和熱門日期區間會吃掉大部分搜尋流量；Polymarket 裡，熱門 market 的 order book 和 price update 會比長尾 market 熱很多。這種情況下，平均 QPS 會騙人，真正需要看的是 hot key、P99 latency 和單一 cache shard 的負載。

## 可以接受多舊的資料？

每個讀取場景都應該有 freshness budget，也就是資料可以落後多久。



- 影片觀看數可以延遲幾分鐘
- 房源描述可以延遲幾十秒到幾分鐘
- 活動場館地址更新後最好很快反映

- 庫存和付款結果通常需要接近即時

這個答案會直接決定 TTL、replication lag 是否可接受、是否能使用 CDN，以及需不需要 cache invalidation。

## 解法地圖

讀取擴展可以用一句話整理：先讓 database 少做浪費的事，再讓讀取分散出去，最後把熱門結果移到更靠近使用者的地方。

常見演進順序如下：

- 先優化單一 database 的讀取路徑：index、query shape、denormalization、materialized view
- 再把讀取分散到更多 database：read replica、sharding
- 再用 cache 避免重複打 database：Redis、Memcached、CDN、edge caching
- 最後處理實戰問題：hot key、cache stampede、invalidation、一致性和觀測

這個順序很重要。很多面試者一開始就畫 Redis cluster，但連資料表該加什麼 index 都沒講。面試官通常會比較喜歡你先把基本盤處理好，再逐步增加複雜度。

## 第一層：把 Database 自己調好

在加入新的基礎設施之前，先確認 database 不是在做沒必要的工作。很多讀取瓶頸其實不是規模太大，而是 query path 太差。

### Index（索引）

Index 本質上是一個方便 database 快速定位資料的資料結構。沒有 index 時，database 可能需要 full table scan，也就是把整張表從頭掃到尾。資料量小時沒感覺，資料量到幾百萬、幾千萬筆後，這會直接變成災難。

有 index 後，database 可以從接近  $O(n)$  的掃描，變成接近  $O(\log n)$  的查找。以一千萬筆資料來說，差異不是「快一點」，而是「能不能用」。

常見要加 index 的地方：

- WHERE 條件常用的欄位，例如 `email`、`user_id`、`status`
- JOIN 用的 foreign key
- ORDER BY 和 pagination 常用的欄位，例如 `created_at`
- 高選擇性的查詢欄位，例如短網址 code、訂單編號

在面試裡，講 index 最好不要只說「我會加 index」。要說清楚加在哪些 query pattern 上。

例如 QR Code Generator 讀取 token：

```
CREATE INDEX idx_qr_codes_token ON qr_codes(token);

SELECT id, owner_id, destination_url, status
FROM qr_codes
WHERE token = 'evt_2026_checkin';
```

訂單列表常見查詢：

```
CREATE INDEX idx_orders_user_created_at
ON orders(user_id, created_at DESC);

SELECT *
FROM orders
WHERE user_id = 123
ORDER BY created_at DESC
LIMIT 20;
```

複合 index 要注意欄位順序。由 `user_id` 和 `created_at` 組成的 index 可以幫助「指定 `user_id` 後依時間排序」的查詢，但不一定能幫助「只依 `created_at` 查詢」的場景。

Index 不是免費的。它會增加寫入成本和儲存空間，因為每次 insert、update、delete 都要維護 index。不過在讀取密集系統裡，設計合理的 index 通常非常值得。更常見的問題不是 index 太多，而是該有的 index 沒有。

## Query Shape

除了加 index，也要看 query 本身是不是太貪心。

常見問題包括：

- 一次 SELECT 太多欄位，實際上只需要列表頁的摘要
- Offset pagination 掃太深，例如 `OFFSET 100000`
- N+1 query，每筆資料又各自查一次作者或統計資訊
- 複雜 join 每次都即時計算
- 排序欄位沒有 index，導致 database 要掃完再 sort

列表頁通常不需要完整物件。你可以把 detail API 和 list API 拆開，列表只回傳必要欄位。Pagination 也可以從 offset-based 改成 cursor-based，避免越翻越慢。

```
SELECT id, title, thumbnail_url, created_at
FROM videos
WHERE channel_id = 123
      AND created_at < '2026-05-01T00:00:00Z'
ORDER BY created_at DESC
LIMIT 20;
```

這類 query 搭配由 `channel_id` 和 `created_at` 組成的 index，通常比深 offset 穩定很多。

## Denormalization（反正規化）

正規化（Normalization）讓資料只存一份，降低更新錯誤與重複儲存。但它也常讓讀取變貴，因為頁面需要的資料被拆在很多表中，每次都要 join 回來。

以 Airbnb booking 摘要頁為例，正規化資料可能長這樣：

```
SELECT g.name, b.check_in, b.check_out, h.title, h.city, h.pr
FROM guests g
JOIN bookings b ON g.id = b.guest_id
JOIN homes h ON b.home_id = h.id
WHERE b.id = 12345;
```

如果這個查詢每秒跑幾千次，join 成本會很可觀。反正規化的做法是建立一張面向讀取的表，例如 `booking_summary`，把頁面需要的欄位預先放好：

```
SELECT guest_name, check_in, check_out, home_title, city, pri
FROM booking_summary
WHERE booking_id = 12345;
```

這張圖用簡化的 user、order、product 表示意 normalized 和 denormalized 的差異；套到 Airbnb 時，可以把 product 想成 home，把 order 想成 booking。代價是寫入變複雜。房源名稱或房東資料更新時，可能要更新多筆摘要資料。但如果這些更新很少發生，booking 摘要讀取非常頻繁，這個 trade-off 就合理。

反正規化的判斷方式可以很務實：

- 如果資料很常被讀、很少被改，適合反正規化
- 如果更新正確性非常重要，反正規化要搭配 transaction、CDC 或背景修補
- 如果資料變動頻繁，重複儲存可能會讓一致性成本太高

## Materialized View 和預先計算

有些讀取慢，不是因為找不到資料，而是每次都要重新算。

例如 Airbnb 房源平均評分：

```
SELECT h.id, AVG(r.rating) AS avg_rating
FROM homes h
JOIN reviews r ON h.id = r.home_id
GROUP BY h.id;
```

如果每次載入房源頁都重新跑一次，讀取成本會隨評論數成長。更好的方式是把結果預先算好，存在 materialized view 或 summary table。

```
CREATE MATERIALIZED VIEW home_ratings AS
SELECT h.id, AVG(r.rating) AS avg_rating
FROM homes h
JOIN reviews r ON h.id = r.home_id
GROUP BY h.id;
```

Enrichment：在真正的系統裡，summary table 常常比 database 內建 materialized view 更彈性。你可以用背景 job、queue consumer 或 CDC pipeline 更新 summary。這也讓你針對不同 freshness budget 設計不同更新頻率，例如評分每 1 分鐘更新，觀看數每 5 秒批次更新。

## 第二層：分散 Database 讀取

當 query 已經合理、index 也正確，但單一 database 還是扛不住，就要讓更多機器分攤讀取。

## Read Replica（讀取副本）

Read replica 是讀取擴展最直覺的 database 手段。所有寫入走 primary，資料複製到多台 replica，讀取則分散到 replica。

典型架構是 leader-follower replication：

- Primary 或 leader 處理寫入
- Replica 或 follower 複製 primary 的資料
- 一般讀取打 replica
- 需要最新資料的讀取打 primary，或打已追上的 replica

Read replica 的好處：

- 增加讀取吞吐量
- 降低 primary 壓力
- 提供容錯能力，primary 掛掉時 replica 可以升級
- 可以把分析查詢或報表查詢導到專用 replica

核心代價是 replication lag。寫入 primary 後，replica 不一定立刻看到新資料。使用者剛更新 profile，下一秒讀到舊 profile，就是 read-after-write inconsistency。

常見處理方式：

- 自己剛寫的資料短時間內讀 primary
- 在 session 裡記錄最後寫入位置，例如 WAL LSN 或 binlog position，只讀已追上的 replica
- 對可接受延遲的資料讀 replica，例如房源瀏覽、market data snapshot、影片 metadata
- 對不能接受延遲的資料讀 primary，例如付款結果、庫存扣減後狀態

面試說法可以是：

我會用 primary 承接寫入，再加多台 read replica 分散讀取。不過這會引入 replication lag，所以使用者剛寫入後的 read-after-write 要特別處理。對使用者剛更新的房源或訂單狀態，我會在短時間內讀 primary；對公開瀏覽頁或影片 metadata，短暫延遲可以接受。

## Sharding（分片）

Read replica 解決的是讀取 QPS，但每台 replica 仍然保有完整資料集。如果資料量本身大到單台 database 的 index、storage 或 working set 都吃不下，就要考慮 sharding。

Sharding 是把資料拆到不同 database。對讀取擴展的幫助有兩個：

- 每個 shard 的資料量變小，query working set 變小
- 讀取負載可以分散到多個 shard

常見分片方式：

- 依 `user_id` hash：適合使用者資料、訂單、個人內容
- 依 `video_id`、`home_id` 或 `market_id` hash：適合內容詳情頁或市場資料
- 依地區分片：適合資料有明顯地域性，且法規或延遲要求不同
- 依功能分片：把 user DB、home DB、order DB 或 market data DB 拆開，隔離不同業務負載

Sharding 的難點不是把資料拆開，而是避免讀取時需要打很多 shard。如果每次請求都要 scatter-gather 到所有 shard，再合併結果，讀取延遲和失敗率都會變差。

所以選 partition key 時，要同時問：

- 資料能不能均勻分散？
- 最常見的讀取請求會打幾個 shard？
- 常被一起讀的資料有沒有放在一起？
- 熱門 key 會不會讓單一 shard 過載？

對大多數讀取擴展問題，sharding 通常不是第一招。它很有用，但運維成本高，資料搬移、跨 shard query、transaction、rebalancing 都會變複雜。若資料量還沒大到單機放不下，cache 和 read replica 往往更划算。

## 第三層：用 Cache 避免重複讀

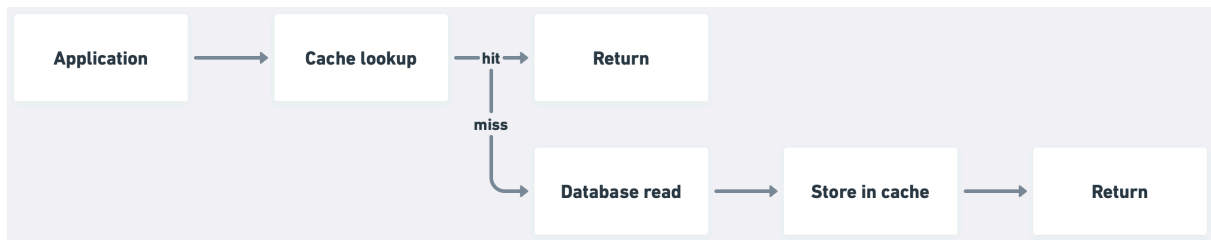
Cache 的核心想法很簡單：如果很多請求都在問同一件事，而且答案短時間內不會改，那就不要每次都問 database。

Database 需要解析 query、查 index、讀資料頁、做 join、排序或聚合。Cache 通常直接從記憶體回傳已經準備好的結果。這不只是延遲較低，也會大幅降低 database 負載。

## Application-Level Cache

Redis 和 Memcached 常放在 application 和 database 中間。

讀取流程通常是 cache-aside：



- Application 先查 cache
- Cache hit，直接回傳
- Cache miss，查 database
- 把結果寫回 cache，下一次就能命中

適合放進 application cache 的資料：

- 熱門房源詳情
- 使用者公開 profile
- 短網址對應
- 影片 metadata
- 首頁模組資料
- 推薦結果的短時間 snapshot

不適合或需要小心的資料：

- 強一致性庫存
- 帳戶餘額
- 權限高度敏感的私人資料
- 每個使用者都不同且很少重複讀的資料

## TTL 和 Freshness Budget

TTL 不是隨便猜一個數字，而是從產品需求推回來。

- 搜尋結果允許 30 秒過時，TTL 可以設 30 秒
- 房源描述允許幾分鐘過時，TTL 可以設 5 分鐘
- 影片觀看數允許最終一致，TTL 可以更長或批次更新
- 活動地址更新後必須很快生效，TTL 就要短，甚至需要主動 invalidation

Enrichment：TTL 最好加 jitter，也就是每個 key 的過期時間加一點隨機偏移。否則一批 key 同時建立，就可能同時過期，造成 cache stampede。

## Cache Invalidation

Cache invalidation 的難點在於資料可能被快取在很多地方：application cache、CDN、browser cache，甚至不同 service 的 local cache。

常見策略：

- TTL：簡單可靠，但會短暫回傳舊資料
- Write-through：寫 database 時同步更新 cache，讀取新鮮，但寫入延遲較高
- Write-around 或 cache-aside：寫入只更新 database，讀取 miss 時才填 cache，實作簡單
- 主動刪除：寫入後刪掉相關 cache key，下一次讀取重建
- Versioned key：cache key 帶版本號，資料更新時換 key
- Tagged invalidation：用 tag 維護一批相關 cache entry，資料變更時一起失效

實務上常常會混用。對關鍵資料用主動失效，對次要資料用 TTL，對熱門資料用背景刷新。

## Stale-While-Revalidate

很多讀取場景並不需要「過期後立刻不能用」。更好的方式是 stale-while-revalidate。

流程是：

- Cache entry 過了 soft TTL，但還沒過 hard TTL
- 使用者請求來時，先回傳舊資料
- 背景非同步刷新 cache
- 刷新完成後，下一個使用者拿到新資料

這個模式可以降低尾端延遲，也能避免熱門 key 同時過期時所有請求都打到 database。對首頁、房源列表、推薦模組很常見。

## Negative Caching

另一個常被忽略的技巧是 negative caching，也就是把「查不到」也快取起來。

例如短網址服務裡，有人用不存在的 code 狂打你的 API。如果每次都查 database，database 仍然會被打爆。可以把「這個 code 不存在」快取短時間，例如 30 秒或 1 分鐘。

要小心的是，negative cache 的 TTL 通常要短。否則如果資料後來被建立，使用者可能在 TTL 內仍然看到不存在。

## 第四層：CDN 和 Edge Caching

CDN 把 cache 從你的 data center 推到全球 edge location。東京使用者不必繞到美國 Virginia 的 origin，直接從東京或附近 edge 拿資料。

CDN 最適合：

- 圖片、影片縮圖、JS、CSS 這類靜態資源
- 公開房源頁
- 公開影片縮圖與 manifest
- 不含個人敏感資訊的 API response
- 搜尋或分類頁的短時間結果

CDN 的價值有兩個：

- 降低使用者 latency
- 讓 origin server 和 database 少承受大量重複讀取

對讀取密集內容，CDN 可以讓 origin 負載下降一個數量級以上。代價是 invalidation 變難，因為資料可能散在很多 edge location。

使用 CDN 時要特別注意 cache key。URL、query string、header、cookie 都可能影響 cache key。如果不小心把 **Authorization** 或個人化 cookie 混進 cache key，hit rate 會掉很低；如果不小心忽略個人化資訊，可能把 A 使用者的資料回給 B 使用者。

原則是：公開且多人共享的內容才積極放 CDN。個人化或敏感資料要非常保守。

## 面試裡怎麼使用這套思路

讀取擴展在面試裡最好的呈現方式，不是列一堆技術名詞，而是沿著最熱的 API endpoint 深挖。

你可以這樣講：

這個系統的瓶頸應該會在讀取端，尤其是 redirect、搜尋列表、影片 metadata 這類高頻 endpoint。我會先確認查詢欄位有正確 index，例如 QR token、home city、created\_at 或 video\_id；接著把讀取量大、變動少的資料放進 Redis 或 CDN。至於庫存、訂單狀態、付款結果這類需要新鮮度的資料，我不會用長 TTL cache，而是拆成獨立讀取路徑，必要時讀 primary 或使用更短的 TTL。

這種說法同時展現了：

- 你知道瓶頸在哪個 endpoint
- 你知道資料新鮮度有差異
- 你沒有過度 cache 強一致性資料
- 你能從 database 優化一路講到 cache 和 CDN

## 常見情境

**QR Code Generator**：`qr_token -> long_url` 是典型一次寫入、多次讀取。

`qr_token` 需要 unique index，避免 redirect 時 full table scan。熱門 token 的 mapping 可以放 Redis；QR image 是靜態資源，可以放 CDN。若某個廣告活動突然爆紅，還要注意單一 token 變成 hot key。

**Airbnb Booking Platform**：房源詳情很適合用 `home_id -> home_object` cache，因為 home data 相對少改。搜尋則比較複雜，會受到城市、日期、價格和排序影響，不能期待所有 query 都有高 hit rate。熱門城市或旺季日期可以短 TTL cache；更通用的搜尋可以靠 database index、PostGIS、Elasticsearch 或 Redis GEO。

**YouTube / 影音平台**：影片檔案、segments、manifest files 和縮圖應該走 object storage + CDN，讓觀看流量不要穿過 application server。影片 metadata、頻道資訊、推薦模組可以放 cache。觀看數和按讚數可以批次更新，接受短暫不一致。

**Polymarket**：Matching Engine 是權威寫入路徑，但讀取端通常會拆成 Market Data Service 和 Order Query Service。Market data 可以由 event stream materialize 到 cache，推播給訂閱者；order query 可以 materialize 到 Order DB，並依 `user_id` sharding，讓單一使用者的訂單查詢落在單一 shard。

**Amazon Price Tracking Service**：價格查詢和商品頁讀取可以利用 cache，但價格變動事件本身應該從 CDC 或 queue 可靠地往下游傳。這種系統要分清楚「最新價格的寫入來源」和「大量使用者查詢價格的讀取模型」。

## 不該優先用讀取擴展的情境

如果系統本身不是 read-heavy，就不要硬套。

- Tesla Robotaxi 位置更新：寫入頻率高，讀取擴展不是第一瓶頸
- 即時協作文件：重點是低延遲同步和 conflict resolution，不是長 TTL cache
- 付款與庫存：可以快取非關鍵展示資訊，但關鍵狀態需要強一致
- 小規模內部工具：單一 indexed database 可能已經足夠，不需要 Redis 和 CDN

## Deep Dive：Hot Key

Hot key 是指大量請求集中在同一個 cache key 或同一筆資料上。

例如 QR Code Generator 裡，某個大型活動把同一個 QR code 印在入口、廣告和直播畫面上，所有掃描都打到同一個 `qr:<token>`。或者 Polymarket 某個熱門市場在重大事件發生時，所有人都在讀同一個 `market:<market_id>`。就算資料在 Redis 裡，單一 Redis shard 仍然有 CPU、網路和連線上限。Cache hit 不代表一定不會掛。



常見解法：

- Request coalescing：同一台 application server 上，對同一個 key 的並發 miss 只發出一個 backend request
- Local cache：在 application process 內短暫快取極熱門資料，減少 Redis 壓力
- Key fanout：把同一份資料複製成多個 key，例如 `qr:abc:bucket:1` 到 `qr:abc:bucket:10`，讀取時隨機選一個
- CDN edge cache：把公開內容推到 edge，避免所有請求回到同一組 cache cluster
- 降級回應：在極端情況下省略非必要欄位，例如暫時不顯示精確 like count

Request coalescing 範例。這裡用 Polymarket 的 market snapshot 當例子：同一台 application server 上，如果已經有人正在重建 `market:<market_id>`，後續 request 直接等待同一個 task，不要各自打 backend。

```
class MarketSnapshotLoader:
    def __init__(self):
        self.pending_by_market = {}

    async def load(self, market_id):
        pending = self.pending_by_market.get(market_id)
        if pending is not None:
            return await pending

        task = asyncio.create_task(read_snapshot_from_store(market_id))
        self.pending_by_market[market_id] = task

    try:
        return await task
    finally:
        self.pending_by_market.pop(market_id, None)
```

Key fanout 的代價是記憶體變多，一致性也更麻煩。更新或失效時，你要處理多份副本。但對會壓垮單一 cache shard 的熱門內容，這個代價通常值得。

# Deep Dive : Cache Stampede

Cache stampede 是熱門 cache entry 同時過期，導致大量請求一起打到 backend。

想像 Airbnb 的首頁推薦或熱門城市搜尋結果每秒十萬次讀取，平常都 cache hit。TTL 到期那一秒，所有請求都 miss，於是大家一起查 database、搜尋引擎或地理索引。你的系統不是被外部攻擊，而是被自己的 cache 過期行為打爆。

常見解法：

- TTL jitter：讓不同 key 不要同時過期
- Distributed lock：只有一個請求負責重建 cache，其他請求等待或回 stale data
- Stale-while-revalidate：先回舊資料，背景刷新
- Probabilistic early refresh：越接近過期，越高機率提前刷新
- Background refresh：對最重要的 key，由排程主動刷新

在面試裡，可以把 distributed lock 當作基本解，但要主動提它的缺點：拿 lock 的請求如果很慢或失敗，其他請求會等待，可能造成 timeout。更成熟的做法通常會搭配 stale data fallback。

## Deep Dive : 資料更新要立刻反映怎麼辦？

最直覺的做法是寫入後刪 cache，但實務上有很多邊界問題：

- 相關 cache key 可能很多，不一定知道要刪哪些
- 刪 Redis 成功，但 CDN invalidation 失敗
- 刪除 cache 後，舊資料又被某個並發請求寫回 cache
- Browser cache 可能仍然留著舊資料

一個常見且可靠的模式是 versioned key。

每筆資料在 database 裡有版本號。更新資料時，在同一個 transaction 裡遞增版本號。讀取時先拿版本號，再組出 cache key。

更新前讀 `home:123:v42`，更新後讀 `home:123:v43`。舊 cache 不需要立刻刪，它只是再也不會被新請求使用，之後靠 TTL 自動清掉。

讀取流程：

- 讀取資料版本號
- 用版本號組 cache key
- 查 cache
- Cache miss 時從 database 讀取，寫入同一個版本 key

寫入流程：

- 開啟 database transaction
- 更新資料
- 遞增版本號
- Commit
- 可選擇預熱新版本 cache

Versioned key 的優點是避免 race condition，也不用廣播刪除訊號。缺點是每次讀取可能多一次 version lookup，舊版本 cache 會累積，所以仍然需要 TTL。

對搜尋結果、推薦列表或複雜聚合資料，versioned key 不一定好用，因為一個結果可能依賴很多資料。這時可以使用 deleted items cache。做法是維護一個小型快取，記錄最近被下架、隱藏或修改的 item id。回傳 Airbnb 搜尋結果或 YouTube 推薦列表前，先用這個小 cache 過濾掉不該出現的內容。這能讓大型 cache 不必立刻全部失效，同時避免使用者看到明顯錯誤的資料。

## 觀測與容量判斷

讀取擴展不能只靠架構圖，還要知道該看哪些數字。

重要指標：

- QPS：總讀取量和各 endpoint 的讀取量
- P50、P95、P99 latency：平均值通常看不出尾端問題
- Database CPU、I/O wait、buffer cache hit ratio
- Slow query log 和 query plan
- Cache hit rate：整體 hit rate 和熱門 key hit rate 要分開看
- Cache eviction rate：是否因記憶體不足一直淘汰
- Hot key distribution：單一 key 或單一 shard 是否過熱
- Replication lag：讀 replica 時資料落後多久
- Origin offload：CDN 擋掉多少 origin request

一個粗略判斷是：如果 database 還在做 full table scan，先修 query 和 index；如果 query 已經很好但 CPU 或 I/O 到頂，考慮 read replica；如果大量請求重複讀相同資料，cache 的收益通常最大；如果全球延遲和 origin 壓力都高，CDN 會很有效。

## 總結

讀取擴展的本質是改變讀取路徑，而不是單純堆技術名詞。

最穩的順序是：先把 database query 做對，包含 index、pagination、query shape 和反正規化；接著用 read replica 分散讀取；如果資料有高度重複讀取，就加 Redis 或 Memcached；如果內容是公開且可共享的，就推到 CDN 或 edge。

真正成熟的設計會把資料分級。不是所有資料都需要同樣的新鮮度，也不是所有資料都值得 cache。影片觀看數可以最終一致，付款結果不能；QR image 可以 CDN，個人帳戶資訊不行；熱門 market data 需要 hot key 保護，長尾資料可能 cache miss 也沒關係。

在面試裡，最有說服力的方式是從具體 endpoint 出發，估算它的讀取量，說清楚資料能接受多舊，然後選擇相應的 index、replica、cache、CDN 和 invalidation 策略。這樣聽起來不像背模板，而是真的在設計一個能活在生產環境裡的系統。

buildmoat.org